

Агенты: ВОЗМОЖНОСТИ и принципы работы

Кодинговые агенты,
контекст, skills и MCP



Мой путь вместе с инструментами

Влад. За несколько лет я прошёл путь от ручного кода до разработки самих агентов.

1

2022 · Школа 21

Писал код руками, сам искал ошибки и документацию.

2

2023 · RUTUBE

Радовались autocomplete в редакторе. Переносили код и ошибки в чат: Ctrl+C, Ctrl+V.

3

2024–25 · A7

Кодинговые агенты уже выполняли целые поручения.

4

2026 · Яндекс

Пришёл разрабатывать агентов и среду для их работы.

Личный путь преподавателя. Эти годы описывают мой опыт, а не даты появления технологий.

Три мысли, к которым будем возвращаться

Наша цель — полезная работа агента в реальном проекте.

1 Мы создаём среду и отвечаем за результат

Даём агенту контекст и инструменты. Проверяем адекватность решения и принимаем работу.

2 Спектр задач гораздо шире автодополнения

Конкурентность, дизайн, исследования и код. Проверяем возможности на задаче с нужным контекстом и обратной связью.

3 Подготовленная среда позволяет писать короче

«Напиши мне тесты» становится понятным поручением, когда агенту доступны требования, примеры и проверки.

Меньше повторяем в промпте, больше полезного контекста хранится в среде.

Широкие возможности не означают гарантии решить любую задачу. Результат зависит и от модели, и от среды.

Что разберём сегодня

1 От редактора к агенту

Как изменился способ работать с моделями

2 Агенты и реальные задачи

Продукты, разработка и мой рабочий процесс

3 Skills

Как передать полезный способ работы

4 MCP

Как подключить данные и действия

5 Среда и результат

Правила, память, исполнители и проверки

Кто переносит контекст и выполняет действия

Обвязка (harness) даёт модели инструменты, контекст и результаты действий. Доступы и правила зависят от среды.

Чат без инструментов

Человек приносит код и ошибки.
Копирует ответ в проект.
Запускает команды и возвращает результат.

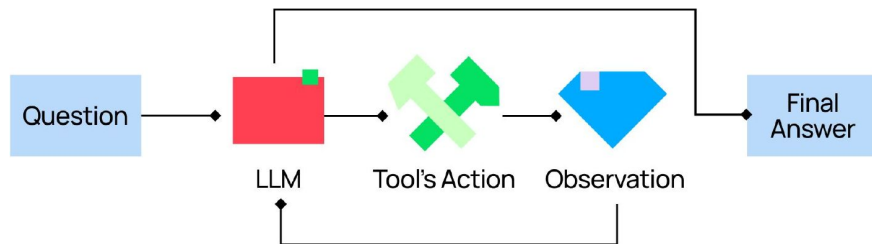
Агентская среда

Агент читает доступные файлы.
Обращается к подключённым системам.
Получает обратную связь и продолжает.

Современный чат тоже может содержать агента. Отличие здесь в действиях, а не в виде окна.

Что происходит внутри агента

Цикл ReAct. Справа — сокращённые события реального учебного запуска Codex CLI.



01 LLM: выбирает действия

Прочитать cache.py и тест. Запустить проверку, ничего не редактируя.

02 Tool: возвращает наблюдение

unittest: 1 test, 1 failure. В t=60 получено старое value-1 вместо value-2.

03 LLM: объясняет причину

Условие now > expires_at пропускает границу. Предложено >=. Файлы не изменены.

Модель выбирает следующий шаг. Harness проверяет права и исполняет tool. Здесь показаны события, не скрытый reasoning.

Агенты вокруг нас

Какие задачи им поручают
и зачем нужны разные агенты.

Сейчас: покупки, создание продуктов и личные помощники.
Затем сосредоточимся на разработке.

Какие примеры сейчас посмотрим

Смотрим, какие данные, действия и ограничения нужны конкретной задаче.

1 Commerce

Разные роли покупателя и продавца

2 v0 и Higgsfield

Продукты вокруг задач создания интерфейсов и медиа

3 OpenClaw и Hermes

Личные помощники с инструментами и памятью

Одна модель, разные commerce-агенты

Агенты могут использовать общее ядро. Различаются инструкции, инструменты, доступные данные и права.

Покупатель

Цель: подобрать товары.

Контекст: каталог и свои заказы.

Действия: поиск, корзина, оформление.

Продавец

Цель: разобраться в продажах.

Контекст: продажи и остатки.

Действия: отчёт и подготовка акции.

Общий пример для двух ролей: commerce agents

Роль в промпте не заменяет проверку доступа. Пример требует интеграции с реальным магазином.

v0 и Higgsfield: продукт вокруг модели

Пользователь получает готовый рабочий процесс: нужные материалы, инструменты и способ оценить результат.

v0

Макет и компоненты проекта помогают получить нужный веб-интерфейс.

Higgsfield

Референсы, модели и медиа-инструменты связаны в процесс производства.

v0

Higgsfield

OpenClaw и Hermes: личные помощники

Общение, доступные инструменты и сохранённые сведения соединяются в одном рабочем процессе.

OpenClaw

Связывает каналы общения с сессиями агента и доступными инструментами.

OpenClaw

Hermes

Использует инструменты, память и сохранённые способы выполнения задач.

Hermes

Память и сохранённые процедуры не переобучают веса модели.

Есть вопросы?

Разным задачам нужны разные данные,
инструменты и ограничения.

Дальше: агенты, которые работают с кодом.

Что могут кодингговые агенты

От небольшого изменения
до исследования и серии PR.

Сейчас: инструменты, реальные кейсы и моя работа.
Затем: как подготовить среду под короткое поручение.

Какие кодинговые агенты встречаются

Изучаем подход. Ведущий может показывать Codex или Claude Code; на практике работаем в OpenCode.

1 Приложения провайдеров

Codex и Claude Code

2 Открытые клиенты

OpenCode и Gemini CLI

3 Минимальный агент и компоненты

Pi: готовый CLI и основа своего приложения

4 Агенты в редакторах

Cursor, VS Code, JetBrains и Zed

OpenCode

Pi

Где работать с агентом

Работа расширилась от терминала до приложений с задачами, review и несколькими сессиями. CLI остаётся полезным.

1 CLI

Поручения и автоматизация из терминала

2 Приложение

Задача, уточнения, preview, diff и продолжение в одном окне

3 Редактор или плагин

Работа рядом с кодом, навигацией и отладчиком

4 ADE

Среда для задач, нескольких агентов и приёмки их работы

5 Сервер или облако

Исполнитель продолжает работу там, где запущен процесс

Форматы могут сочетаться. ADE — название класса сред, а не единый стандарт.

За что платим: подписка или API

API оплачивается отдельно от подписки. Пример справа — один вызов; задача агента обычно включает несколько.

Subscription-based

ChatGPT Plus с Codex: \$20/мес.

Claude Pro с Claude Code:
\$20/мес.

Включённое использование с
лимитами.

Тарифы подписки

API-based

GPT-6 Sol, за 1 млн токенов:

\$2 входных / \$10 выходных.

100K входных + 10K выходных =
\$0.30.

API: GPT-6 Sol

29.09.2026, USD, месячные планы; API Standard $\leq 272K$ input. Пример за один вызов, без кэша и tools. Налоги отдельно.

Токены: одинаковый смысл, разная длина

Модель разбивает текст на токены. Это могут быть слова, их части, пробелы и знаки.

Английский · 10 токенов

Write tests for the API
and check boundary cases.

Русский · 14 токенов

Напиши тесты для API
и проверь граничные случаи.

[Попробовать свой текст в tokenizer](#)

Один пример в OpenAI Tokenizer, GPT-5.x & O1/3. Соотношение зависит от текста и tokenizer, перевод не всегда выгоднее.

Spotify: среда для инженеров и агентов

По данным Spotify, больше 99% инженеров используют AI для кода каждую неделю. Общая среда помогает людям и агентам.

СРЕДА

- 1 Backstage**
Контекст о сервисах
- 2 Fleetshift**
Массовое поручение
- 3 Honk**
Исполнение в репозитории
- 4 PR и проверки**
Приёмка команды

240 PR. ≈10 инженерных недель вручную — оценка Spotify.

Spotify: контекст через MCP и CLI

Самоотчёт Spotify. Отдельные Agent SkillStore и MCPStore публичными источниками не подтверждены.

КАК УСТРОЕНА РАБОТА

- 1 Backstage**
Каталог сервисов, владельцы и документация. Возможности доступны агентам через MCP и CLI.
- 2 Fleetshift + Honk**
Выбор репозитория, постановка миграции, исполнение и наблюдение за прогрессом PR.
- 3 Команды-владельцы**
Тесты, review и приёмка. При отсутствии автоматических проверок нужна ручная валидация.

Кейс миграции: Honk part 4

Bitrise: агент ведёт изменение несколько дней

Кейс Bitrise: около 200 использований компонента в 167 файлах; PR сопровождался четыре дня.

1

Задача в Slack или Jira

Колега ведёт поручение и передаёт кодирование исполнителю в изолированном окружении.

2

Продолжение по событиям

CI и комментарии review возвращают агента к работе. Состояние живёт дольше одного ответа.

3

Решение человека

Человек проверяет результат и разрешает merge. Агент сам это разрешение не выдаёт.

Bitrise: устройство Колега и кейсы

Июль 2026. Кейс и цифры опубликованы Bitrise, независимого сравнения здесь нет.

Как изменилась моя работа

Моя оценка: команда проводит не больше 1% рабочего времени в редакторе. Основная работа идёт через агентские приложения.

Задача	Раньше	Сейчас
Обсудить интерфейс	Ждал макет	Собираю интерактивный прототип
Изменить соседний сервис	Ждал оценки команды	Исследую код и предлагаю PR
Разобрать метрики	Вручную изучал панели	Поручаю поиск связей и проверяю данные
Собрать картину задачи	Переключался между системами	Соединяю wiki, тикеты, код и метрики

Личный опыт Влада, не статистика отрасли. Код и последствия изменений всё равно проверяем.

Растут и модели, и среда вокруг них

Выбор модели меняет опыт работы с тем же агентом: от локальной правки до исследования и сложного изменения.

Модель

Понимает более сложную задачу.
Связывает найденные факты.
Предлагает следующий шаг.

Harness

Даёт инструменты и контекст.
Сохраняет состояние работы.
Возвращает ошибки и проверки.

Сильная модель и хорошая среда дополняют друг друга. Возможности проверяем на своей задаче.

Модель выбираем под задачу

Codex и Claude Code — приложения, в которых выбирают модель. Ищем достаточное качество по разумной цене.

Задача	Как начать	Примеры моделей
Небольшая ясная правка, извлечение данных	Быстрая модель и готовая проверка	GPT-6 Luna Claude Haiku 4.5 Gemini 3.5 Flash-Lite
Фича по контракту, тесты, обычное review	Повседневная модель с подходящим effort	GPT-6.1 Sol Claude Sonnet 5.5 Gemini 3.8 Flash
Неясный баг, архитектура, большое исследование	Сильная модель и больше времени	GPT-6 Astra Claude Opus 5.5 Claude Fable 5.1

[AIMEC: как читать сравнения](#)

[Claude: модели](#)

[Gemini: модели](#)

Benchmark измеряет модель, harness и условия задачи вместе.

Язык и стек влияют на работу агента

Слева — авторская оценка, не benchmark моделей. Это повод обсудить различия и проверки на своём проекте.

ЯЗЫК	AI-ДРУЖЕЛЮБНОСТЬ
Python	 9/10
TypeScript	 8.5/10
Go	 8/10
Rust	 5.5/10 gen, 9/10 надёжность
Java	 7.5/10
C/C++	 3.5/10

АВТОРСКАЯ ОЦЕНКА, НЕ BENCHMARK МОДЕЛЕЙ

1 Данные и экосистема

Примеры кода и документация помогают. Точный состав обучения закрытых моделей неизвестен.

2 Инструменты проверки

Компилятор, type checker, тесты, race detector и sanitizer проверяют разные свойства.

3 Выбор под свой проект

Сравниваем модели на знакомых задачах и версиях библиотек. Старые оценки не переносим автоматически.

Редкий стек: добавьте контекст, примеры и проверки.

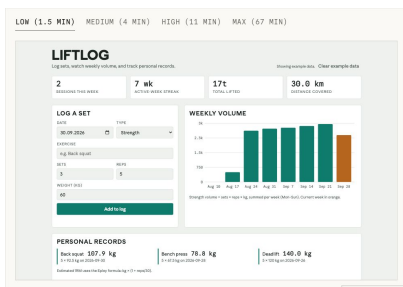
[MultiPL-E: разные языки](#)

[SWE-bench Multilingual](#)

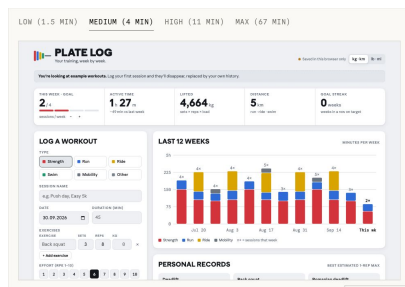
Одна задача, четыре уровня effort

«Сделай приложение для тренировок». Иллюстративные запуски автора Anthropic на Opus 5.5.

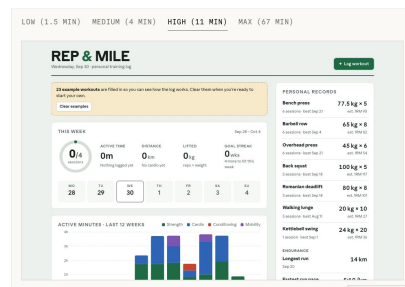
Low · 1.5 мин



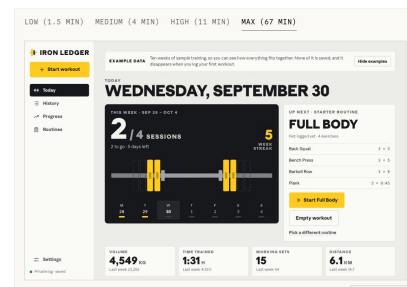
Medium · 4 мин



High · 11 мин



Max · 67 мин



Low / medium: выбираем направление

Первый код, обсуждение и короткие итерации. Сверяем с человеком, ту ли задачу решаем.

High: проверяем результат

Тесты, review и поиск крайних случаев после согласования смысла задачи.

Max: сложная автономная работа

Даём больше времени. Заранее определяем цель, границы и критерий завершения.

Больше effort: больше проверок и самостоятельных решений. Не обязательно ближе к вашей цели.

«Напиши мне тесты»: один запрос, разная среда

Короткая инструкция работает, когда недостающие подробности можно получить из проекта и доступных источников.

Проект не объяснён

Какое поведение правильное?
Какие тесты уже есть?
Чем их запускать и что можно
менять?

Контекст подготовлен

Есть контракт и удачные
примеры.
Известны framework и команды.
Заданы ограничения и
критерий готовности.

Учебное сравнение. Если требование неизвестно, агенту нужно уточнение, а не догадка.

Контекст и длина ответа — разные ограничения

Модель получает материалы, обрабатывает их и генерирует ответ. У каждого шага есть ограничения.

КОНТЕКСТ

Что модель читает

Инструкции, история разговора, файлы и результаты инструментов.

Пример: большая папка документов

В контекст добавляем то, что помогает задаче. Большая история не гарантирует точного ответа.

Шум, противоречия и устаревшие сведения увеличивают риск ошибки. При смене задачи помогают сводка и новый чат.

ДЛИНА ОТВЕТА

Что модель пишет

Один ответ имеет собственный предел. Даже большой контекст не делает ответ безграничным.

Пример: короткий отчёт по папке

Есть вопросы?

Короткому поручению нужна подготовленная среда.
Начнём со способа выполнять работу.

Дальше: skills: инструкции, примеры и исполняемый код.

Skills: полезный способ работы

Один раз описали процедуру,
потом применяем её к новым задачам.

Сейчас: понятный пример, устройство пакета и готовые skills.
Затем: подключение внешних систем через MCP.

Что передаёт skill

Skill объясняет, когда применять процедуру, что сделать и как проверить результат.

Skill объясняет способ выполнить работу.

1

Когда использовать

Описание помогает выбрать подходящий сценарий.

2

Как выполнить

Шаги, справки, примеры и при необходимости скрипты.

3

Что вернуть

Ожидаемый результат, проверки и честные ограничения.

Источник: agentskills.io

Реальный skill: воспроизвести баг в n8n

n8n публикует процедуру reproduce-bug для разработки самого n8n. На входе — полное описание ошибки из тикета.

1

Разобрать жалобу

Установить ожидаемое поведение и условия воспроизведения.

2

Найти нужный слой

Выбрать unit, integration или E2E по месту дефекта и правилам проекта.

3

Написать тест

Воспроизвести проблему и получить ожидаемое падение.

4

Вернуть доказательство

Показать тест, команду и причину падения. Сам продукт этот skill не исправляет.

[Открыть reproduce-bug / SKILL.md](#)

Ценность процедуры: одинаковые вход, границы и результат. Это skill для кода n8n, не конструктор пользовательских workflow.

Skill может содержать тысячи строк кода

Реальный установленный пакет SkillStore v23.0.0: каталог, версии, доставка skills и диагностика.

Инструкция направляет агента. Основную логику выполняют скрипты.

1

235 строк

SKILL.md: когда применять, какой справочник читать и какую команду выбрать.

2

7 378 строк

10 собственных скриптов: API-клиент, авторизация, диагностика и адаптеры платформ.

3

1 247 строк

12 других Markdown-файлов: инструкции и справки по отдельным задачам.

Спецификация: scripts, references и assets

Локальный снимок 27.09.2026. Физические строки с комментариями и пустыми строками, без зависимостей.

Анатомия skill и загрузка в контекст

Учебный фрагмент. Пакет лежит на диске целиком, в контекст попадает нужная часть по мере работы.

ПАКЕТ SKILL

SKILL.md

```
--  
name: reproduce-bug  
description: Reproduce a bug with a failing test  
--
```

```
# Procedure  
Read the contract and project examples.  
Write a regression test.  
Run it and report the failure.
```

references/ подробные правила
scripts/ исполняемый код
assets/ шаблоны и данные
Как это делает OpenCode

ЗАГРУЗКА В КОНТЕКСТ

1

При выборе

name и description помогают найти подходящий skill.

2

После активации

Подгружается основная инструкция SKILL.md.

3

По необходимости

Читаются справки. Скрипт выполняется; в контекст приходит его результат.

Пример исполняемого skill

Типовая progressive disclosure. Точный способ обнаружения и загрузки задаёт клиент; весь каталог не обязан попадать в prompt.

Что написать в хорошем skill

Вход: требование, diff, примеры тестов.

Шаги: прочитай контракт → выбери случаи → проверь.

Границы: не меняй продукт ради зелёного теста.

Проверка: команда, exit code и фактический результат.

Ответ: что проверено, что сломано, что неизвестно.

На примере тестов: у процедуры есть вход, границы и наблюдаемый результат.

Как писать и проверять skills

Инструкцию подкрепляем permissions и проверками. Сам текст не обеспечивает запрет.

Готовый скрипт или временный код через SDK

SDK даёт больше комбинаций действий. Версии, авторизацию, права и сгенерированный код всё равно проверяем.

Готовые скрипты

Заранее реализуем нужные операции.

Проще повторять и тестировать.

Новые сценарии требуют доработок.

Инструкция по SDK

Показываем модули, примеры и ограничения.

Агент пишет небольшой код под задачу.

Проверяет и выполняет его.

Общий источник для двух подходов: Anthropic

Внутренние skills меняют повседневную работу

В большой компании у каждой системы свои данные, API, правила доступа и рабочие процедуры.

Трекер, wiki, код, аналитика

Skills объясняют принятый порядок работы.

API и инструменты дают доступ.

Агент соединяет шаги в поручение.

Внутренний каталог

Команда один раз описывает процедуру.

Владелец поддерживает версии и примеры.

Коллеги находят и переиспользуют skill.

Личный опыт Влада. Наличие внутреннего API или каталога само по себе не гарантирует качество результата.

Skills: текст, код, тесты и интерфейс

Пять конкретных процедур. Имя ведёт к исходному SKILL.md.

- | | | |
|---|--|--|
| 1 | <u>Humanizer-ru</u> | Отредактировать русский README без штампов. |
| 2 | <u>Caveman</u> | Получать короткие технические ответы. |
| 3 | <u>test-driven-development</u> | Пройти цикл: падающий тест, исправление, проверка. |
| 4 | <u>webapp-testing</u> | Проверить локальный UI через Playwright. |
| 5 | <u>frontend-design</u> | Продумать композицию и реализовать интерфейс. |

10 примеров skills · 1/2

[Каталог skills.sh](#)

[Каталог OpenAI Skills](#)

[Каталог Anthropic Skills](#)

Для integrations нужны инструменты и авторизация. Читайте инструкции, зависимости и лицензию до запуска.

Skills: рабочие системы и документы

Публичные аналоги знакомых задач: PR review, трекер и wiki.

6	<u>gh-address-comments</u>	Разобрать и исправить выбранные замечания GitHub review.
7	<u>linear</u>	Работать с задачами и проектами через Linear MCP.
8	<u>notion-knowledge-capture</u>	Превратить обсуждение в wiki, ADR или FAQ.
9	<u>docx</u>	Создать документ с правками, комментариями и проверкой.
10	<u>agents-sdk</u>	Разработать агента на Cloudflare SDK: state, schedule, MCP.

10 примеров skills · 2/2

[Каталог skills.sh](#)

[Каталог OpenAI Skills](#)

[Каталог Anthropic Skills](#)

Для integrations нужны инструменты и авторизация. Читайте инструкции, зависимости и лицензию до запуска.

Свою процедуру можно сначала надиктовать

«Тесты пишем по контракту.

Сначала посмотри существующие примеры.

Не меняй продукт ради зелёной проверки.

В конце покажи, что действительно запустилось».

Superwhisper превращает речь в текст. Этим способом можно поставить задачу или набросать процедуру для skill.

Открыть: superwhisper.com

Диктовка сама по себе не является агентом. Проверяем числа, термины и частицу «не».

Есть вопросы?

Skill сохраняет способ работы.
Пользу подтверждает результат на задаче.

Дальше: MCP: как подключать инструменты и данные.

МСР: подключение внешних возможностей

Разработан Anthropic.
Открытый стандарт с 25 ноября 2024 года.

Дальше: связь с API, примеры, подключение и отладка.
Затем: настройки самого кодингового агента.

MCP даёт агентским приложениям общий интерфейс

API задаёт операции сервиса. MCP — общий стандарт их обнаружения и вызова. Сообщения используют JSON-RPC 2.0.

ЦЕПОЧКА ВЫЗОВА

1 Агентское приложение

выбирает вызов

2 MCP-сервер

описывает и исполняет tool

3 Сервис / API

выполняет операцию

ЧТО ДАЁТ ПРОТОКОЛ

1 Переиспользование

Один сервер могут подключать разные совместимые клиенты.

2 Обнаружение возможностей

Клиент получает tools и их схемы в согласованном формате.

3 Сообщения JSON-RPC 2.0

Единый формат запросов, результатов и ошибок между клиентом и сервером.

«Создай задачу»: `tool create_issue(project, title)` внутри MCP может вызвать `POST /issues` обычного API.

Архитектура MCP

MCP часто работает поверх API. Для одной интеграции прямой SDK или CLI тоже подходит; права MCP сам не выдаёт.

Доклад: MCP под капотом

Как skill и MCP работают вместе

Skill может использовать MCP, CLI или SDK. Выбор зависит от доступных интерфейсов и способов сопровождения.

Skill: процедура

Прочитай требования.
Найди подходящие примеры.
Выполни проверки и составь
отчёт.

MCP: подключение

Получить задачу из трекера.
Прочитать обсуждение PR.
Проверить сценарий в
браузере.

Авторизация возможна и в CLI/SDK. MCP не выдаёт права сам и не заменяет правила работы.

Полезные MCP для разных задач

Выбираем подключение под задачу. Название ведёт к инструкции автора.

GitHub

Issues, pull requests и обсуждения review.

Playwright

Сценарии в браузере и проверка интерфейса.

Context7

Документация и примеры нужного SDK.

Figma

Макеты, компоненты и дизайн-контекст.

Официальный MCP Registry

Проверьте совместимость клиента, авторизацию и права. Для экспериментов с БД используйте тестовый проект и read-only.

Sentry

Ошибки, stack traces и диагностика проблем.

Supabase

Схема БД, запросы и логи проекта.

Notion

Поиск и работа со страницами workspace.

Linear

Задачи, проекты и их актуальное состояние.

Общие интеграции и корпоративные каталоги

Команда может централизованно подключать внутренние системы и сопровождать доступы.

1

Выбрать возможности

Трекер, wiki, репозитории и аналитика: только нужные операции.

[MCP Registry](#)

2

Настроить авторизацию

Учётные данные, разрешения и ограничения проверяются вне текста промпта.

3

Подключить клиент

Каталог или портал помогает найти разрешённые инструменты и их версии.

[Cloudflare MCP portals](#)

Пример платформы: Cloudflare MCP server portals. Права в исходной системе проверяются отдельно.

Как написать и проверить свой MCP

Начинаем с одного полезного инструмента, затем проверяем его отдельно и в агенте.

1

Реализовать инструмент

Описать tool, аргументы и результат. Написать обработчик через SDK протокола.

[MCP Inspector](#)

2

Проверить в Inspector

Подключиться, передать известные аргументы, изучить результат и ошибку.

[Учебная запись MCP](#)

3

Проверить в агенте

Посмотреть выбранный tool и убедиться в результате в целевой системе.

Подключённый MCP ещё не значит использованный. Проверяем tool call и реальный результат в целевой системе.

Есть вопросы?

Skill задаёт способ работы.
MCP подключает возможности.

Дальше: правила, память и проверки внутри агентской среды.

Как подготовить среду кодингового агента

Контекст, доступы, память
и обратная связь под ваш проект.

Сейчас: правила, проверки и организация работы.
В конце вернёмся к «Напиши мне тесты».

Правила проекта и отдельных директорий

Правила уже лежат в крупных публичных репозиториях: Ruff, Next.js и Cloudflare Workers SDK.

Общие правила

AGENTS.md или CLAUDE.md.
Где контракт и примеры.
Как проверять и что нельзя
менять.

Ruff: AGENTS.md

Next.js: AGENTS.md

Локальные правила

Инструкции для backend или
frontend.
Вложенный CLAUDE.md либо
.claude/rules/*.md с paths.

Workers SDK: AGENTS.md

rules.md не универсальное имя для всех агентов. В новой сессии проверяем применение правил.

Hooks и проверки дают обратную связь

Debug loop: проверяем гипотезу наблюдением. Feedback loop: автоматически возвращаем результат проверки после правки.

1

Гипотеза и наблюдение

Агент предполагает причину. Тест, лог или воспроизведение проверяет её.

2

Правка и событие

После изменения hook запускает заданную команду: lint, typecheck или tests.

3

Обратная связь

Exit code и ошибка возвращаются агенту. Он исправляет причину и повторяет проверку.

Slash-command запускает пользователь. Агентские hooks и Git pre-commit — разные механизмы.

Разрешения и отмена действий

Чем больше действий доступно агенту, тем важнее понимать их последствия.

1 Permissions и auto-approve

Plan/read-only помогает исследовать. В режиме правок ограничиваем shell, tools и внешние операции; auto-approve меняет подтверждения.

2 Rewind и checkpoints

Удобны для возврата файлов или разговора в поддерживаемом клиенте.

3 Внешние изменения

Отправленное сообщение или запись в БД могут не откатиться вместе с файлами.

Рутина для агента: объяснить diff, предложить commit message, проверить изменение перед commit.

Память, compaction и поиск

Compaction сжимает историю. Память сохраняет знания между сессиями. Поиск возвращает нужные факты сейчас.

1 Текущий разговор

Claude Code и Codex умеют auto-compaction. Сводка экономит место, но может потерять детали. Для новой задачи делаем handoff.

2 Долгая память

Memory bank, правила и решения живут вне чата. Сохраняем источник, дату и ограничения; обновляем устаревшее.

3 Поиск в проекте

rg — текст; AST/LSP — символы и связи; RAG — релевантные фрагменты. В контекст попадает результат поиска.

Claude: context management

Codex: auto compaction

Agent Home: память

Prompt cache может удешевить повторный вход, но не освобождает контекстное окно.

Несколько чатов и субагенты

Для большого контекста выделяем исследование в отдельное поручение. Для параллельных правок сначала разделяем файлы.

Отдельный чат

Другая задача или независимый взгляд.
Человек переносит исходные данные и принимает результат.

Субагент

Основной агент выдаёт подзадачу:
найти код, написать тест, проверить diff.
Получает результат и продолжает работу.

Контекст субагента зависит от клиента. Ни чат, ни субагент сами по себе не создают отдельный worktree.

Worktree разделяет файлы, но не всю систему

Два агента в общей папке могут конфликтовать. Worktree разделяет файлы; порты, БД и общий API могут конфликтовать.

Worktree A

Агент меняет API в своей рабочей папке.

Worktree B

Агент меняет интерфейс в другой рабочей папке.

Общий источник для двух worktree: Git

Ветка сама по себе не создаёт второй рабочий каталог.

Длительные поручения: цель и условия остановки

Между итерациями сохраняем состояние в файлах и Git. Продолжаем до результата, лимита или повторяющейся блокировки.

/goal в Codex

Цель и критерий готовности.
Работа продолжается в рамках цели.
Бюджет и границы действий.

Codex: goals

Ralph-подобный цикл

Цикл запускает итерацию.
Агент читает прогресс.
Проверка решает, идти ли дальше.

Долгие задачи: harness

Цикл — программа; выбор следующего действия — модель. Конкретный механизм продолжения зависит от реализации.

Пример: reviewer с контекстом проекта

В моём рабочем процессе reviewer полезен благодаря дополнительному контексту, а не одной просьбе «проверь код».

1

Получить основания

Diff, задача, контракт и связанные вызовы.

2

Проверить гипотезу

Найти проблемный сценарий и подтвердить его кодом или тестом.

3

Вернуть замечание

Объяснить условие ошибки и последствие. Человек принимает решение.

Личный опыт Влада. Самоуверенный текст reviewer не является доказательством дефекта.

Есть вопросы?

Среда задаёт правила и обратную связь.
Ответственность за приёмку остаётся у нас.

Дальше: соберём всё на коротком поручении про тесты.

За результат отвечаем мы

AI-агент — инструмент. Люди выбирают цель и принимают решение о выпуске.

1 Что разрабатывать

Выбираем нужную фичу и требования. Агент помогает исследовать и реализовать решение.

2 Когда остановиться

Задаём критерии готовности и прекращаем бесконечное „сделай ещё лучше“.

3 Что выпускать

Оцениваем архитектуру, проверяем поведение и риски. Решение о готовности принимаем по доказательствам.

Исполнение можно делегировать. Приёмка и ответственность остаются у людей.

Ошибка беспилотника может навредить людям. Проверка должна соответствовать цене ошибки.

AI — рычаг для ваших навыков

Мы продолжаем управлять процессом на более высоком уровне: через цели, вопросы, контекст и проверку.

1 Поставить задачу

Выбрать цель и критерий результата. Задать вопрос, который выявит неверное допущение.

2 Организовать команду агентов

Дать нужный контекст, разделить работу и проверить, как части складываются в систему.

3 Принять результат

Оценить архитектуру и последствия. Решить, когда результат достаточно хорош и пора остановиться.

Умение руководить работой превращает агентов в вашу команду.

Начинающему AI тоже помогает учиться. Без понимания результата легко масштабировать ошибки.

Что посмотреть и почитать дальше

Среда для агента и проверка результата

Мы создаём среду для агента и проверяем адекватность того, что он выдаёт.

Что нужно	Чем обеспечиваем	Как проверяем
Понимать проект	Контекст, правила, память	Нужный источник найден и применён
Знать способ работы	Skills и примеры	Процедура подходит задаче
Действовать	CLI, SDK и MCP	Получен реальный результат
Вести сложное поручение	Чаты, субагенты, worktree	Понятны границы и прогресс
Принять изменение	Тесты, hooks и review	Выполнено исходное требование

Вопросы? На практике применим эти принципы в OpenCode.